FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CCS007L
(COMPUTER PROGRAMMING 2)

EXERCISE

7

FILE HANDLING

Student Name / Group Name:	**Jed L. Hizon**	
Members (if Group):	**Name**	**Role**
Section:	**AN11 – Term 2**	
Professor:	**Mr. Cezar**	

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design, implement and evaluate computer-based systems or applications to meet desired needs and requirements. [PO: C]

II. COURSE LEARNING OUTCOME/S (CLO)ADDRESSED BY THE LABORATORY EXERCISE

- Apply the fundamental principles of handling CString values, pointers and memory allocation, and structures using C++in solving computing activities [CLO: 2]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE At the end of this exercise, students must be able to:

- Create a program that open a given text file, process, and write the output to another file.
- Create a record management system that stores basic information on a given text file and load the data when opened.

IV. BACKGROUND INFORMATION

To perform file processing in C++, header files <iostream> and <fstream> must be included in your C++ source file.

This requires another standard C++ library called fstream, which defines three new data types:

Data Type	Description
ofstream	This data type represents the output file stream and is used to create files and to write information to files.
ifstream	This data type represents the input file stream and is used to read information from files.
fstream	This data type represents the file stream generally, and has the capabilities of both ofstream and ifstream which means it can create files, write information to files, and read information from files.

V. GRADING SYSTEM / RUBRIC

Trait	(Excellent)	(Good)	(Fair)	(Poor)
Requirement Specification(30pts)	Able to identify correctly all input and output and provide alternative. **(28-20pts)**	Able to identify correctly all input and output **(25-17pts)**	Able to identify only one input or output **(22-14pts)**	Unable to identify any input and output **(20-11pts)**
Data type(20pts)	Able to apply required data type or data structure and produce correct results **(1820pts)**	Able to apply required data type or data structure and produce partially correct results **(15-17pts)**	Able to identify required data type or data structure but does apply correctly **(12-14pts)**	Unable to identify required data type **(9-11pts)**
Input Validation(20pts)	The program works and meets all specifications. Does exception al checking for errors and out-of- range data **(18-20pts)**	The program works and meets all specifications. Does some checking for errors and out of range data **(15-17pts)**	The program produces correct results but does not display correctly Does not check for errors and out of range data **(12-14pts)**	The program produce s incorrect results **(9-11pts)**
Free from syntax, logic, and runtime errors (10pts)	Unable to run program **(10pts)**	Able to run program but have logic error **(8-9pts)**	Able to run program correctly without any logic error and display inappropriate output **(6-7pts)**	Able to run program correctly without any logic error and display appropriate output **(5pts)**
Delivery (10pts)	The program was delivered on time **(10pts)**	The program was delivered after 5 minutes from the time required. **(89pts)**	The program was delivered after 10 minutes from the time required. **(67pts)**	The program was delivered after 15 (or more) minutes from the time required. **(5pts)**
Use of Comments (10pts)	Specific purpose is noted for each function, control structure, input requirements, and output results. **(10pts)**	Specific purpose is noted for each function and control structure. **(8-9pts)**	Purpose is noted for each function. **(67pts)**	No comments included. **(5pts)**

VI. LABORATORY ACTIVITY

INSTRUCTIONS:
Copy your source codes to be pasted in this document as well as a screen shot of your running output.

ACTIVITY 7.1: Player's record with file operations

Redo your program in Laboratory Activity 6, this time you have to save the records to a text file and will be retrieved for display. Your program must be able to keep records and compute for the scores of 5 players. The information of each player contains: Nickname, Age and two best played scores.

The program will prompt the user to choose the operation of records from a menu as shown below:

```
==========================================
                MENU
==========================================
```

1. Add record
2. View players records
3. Compute for the average
4. Show the player(s) who gets the max average.
5. Show the player(s) who gets the min average.
6. Open the file.
7. Close the File
8. Exit

Follow the following steps to help you complete the task

1. Declare the header files

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;
```

2. Create a Player and Node structure

```
struct Player {
   string nickname;
   int age;
   double scores[5];
};

struct Node {
   Player player;
   Node* next;
};
```

3. Set head of the linked list

```
Node* head = nullptr;
```

4. Let's create a Function to add a player record to the linked list

```
void addRecord(const Player& player) {
      Node* newNode = new Node;
      newNode->player = player;
      newNode->next = head;
      head = newNode;
   }
```

5. Let's create a Function to view player records

```
void viewRecords() {
   Node* current = head;
   while (current != nullptr) {
      cout << "Nickname: " << current->player.nickname << endl;
      cout << "Age: " << current->player.age << endl;
      cout << "Scores: ";
      for (int i = 0; i < 5; ++i) {
         cout << current->player.scores[i] << " ";
      }
      cout << endl;
      cout << "----------------------------------" << endl;
      current = current->next;
   }
}
```

6. The function to compute the average of scores

```
   void   computeAverage()   {
      Node*   current   =   head;
      while (current != nullptr) {
         double sum = 0;
         for (int i = 0; i < 5; ++i) {
            sum += current->player.scores[i];
         }
         double average = sum / 5.0;
         cout << "Nickname: " << current->player.nickname << endl;
         cout << "Average Score: " << average << endl;
         cout << "----------------------------------" << endl;
         current = current->next;
      }
   }
```

7. Here's the function of geting the maximum average score

```
void showMaxAverage() {
   Node* current = head;
   double maxAverage = -1; // Initialize to a non-valid value
   while (current != nullptr) {
      double sum = 0;
      for (int i = 0; i < 5; ++i) {
         sum += current->player.scores[i];
      }
      double average = sum / 5.0;
      if (average > maxAverage) {
         maxAverage = average;
      }
      current = current->next;
   }
   // Now, iterate again to display players with the max average
   current = head;
   while (current != nullptr) {
      double sum = 0;
      for (int i = 0; i < 5; ++i) {
         sum += current->player.scores[i];
      }
      double average = sum / 5.0;
      if (average == maxAverage) {
         cout << "Nickname: " << current->player.nickname << endl;
         cout << "Average Score: " << average << endl;
         cout << "---------------------------------" << endl;
      }
      current = current->next;
   }
}
```

8. Here's the function for geting the minimum average

```
void showMinAverage() {
    Node* current = head;
   double minAverage = 100; // Initialize to a value higher than the possible average
   while (current != nullptr) {
      double sum = 0;
      for (int i = 0; i < 5; ++i) {
         sum += current->player.scores[i];
      }
      double average = sum / 5.0;
      if (average < minAverage) {
         minAverage = average;
      }
      current = current->next;
   }
   // Now, iterate again to display players with the min average
   current = head;
   while (current != nullptr) {
      double sum = 0;
      for (int i = 0; i < 5; ++i) {
         sum += current->player.scores[i];
      }
      double average = sum / 5.0;
      if (average == minAverage) {
         cout << "Nickname: " << current->player.nickname << endl;
         cout << "Average Score: " << average << endl;
         cout << "---------------------------------" << endl;
      }
      current = current->next;
   }
}
```

9. Now we need an open and close function

```
void openFile() {
   ofstream outFile("player_records.txt");
   Node* current = head;
   while (current != nullptr) {
      outFile << current->player.nickname << endl;
      outFile << current->player.age << endl;
      for (int i = 0; i < 5; ++i) {
         outFile << current->player.scores[i] << " ";
      }
      outFile << endl;
      current = current->next;
   }
   outFile.close();
   cout << "Records saved to 'player_records.txt'." << endl;
}

// Function to indicate closing the file
void closeFile() {
   cout << "File closed." << endl;
}
```

10. Complete the program by adding the main function creating a menu. Use looping and switch statements for the main function.

11. Submit the file and output screenshots of your program here:

```
==================================
              MENU
==================================

1. Add Record
2. View players record
3. Compute for the average
4. Show the player(s) who get the max average
5. Show the player(s) who get the min average
6. Open the file
7. Close the File
8. Exit
Enter Choice:
```

```
===Add Player Record===
-----------------------
Enter Player Name: Jed
Enter Player Age: 18
Enter Score 1: 10
Enter Score 2: 8
Enter Score 3: 9
Enter Score 4: 12
Enter Score 5: 7
Press any key to continue . . .
```

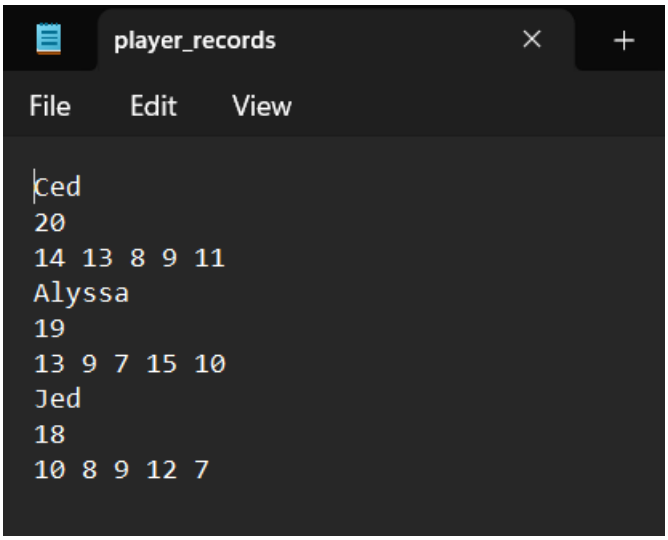
```
===View Player Record===
------------------------
Nickname: Ced
Age: 20
Scores: 14 13 8 9 11
--------------------------------
Nickname: Alyssa
Age: 19
Scores: 13 9 7 15 10
--------------------------------
Nickname: Jed
Age: 18
Scores: 10 8 9 12 7
--------------------------------
Press any key to continue . . .
```

```
===Show Max Average===
------------------------
Nickname: Ced
Average Score: 11
--------------------------------
Press any key to continue . . .
```

```
===Compute Player Average===
-------------------------
Nickname: Ced
Average Score: 11
--------------------------------
Nickname: Alyssa
Average Score: 10.8
--------------------------------
Nickname: Jed
Average Score: 9.2
--------------------------------
Press any key to continue . . .
```

```
===Show Min Average===
------------------------
Nickname: Jed
Average Score: 9.2
--------------------------------
Press any key to continue . . .
```

```
===Open File===
------------------------
Records saved to 'player_records.txt'.
Press any key to continue . . .
```




```
===Close File===
------------------------
File closed.
Press any key to continue . . .
```

```
Program Ended. Thank You!

--------------------------------
Process exited after 46.91 seconds with return value 0
Press any key to continue . . .
```

VII. QUESTION AND ANSWER

Briefly answer the questions below. Avoid erasures. For group activity, specify the name of GROUP MEMBER/s who answered the question. Do not forget to include the source for all NON-ORIGINAL IDEAS.

• What C++ header file libraries are used in file handling? The primary header file library used in C++ for file handling is <fstream>.
• What are file modes? Discuss each mode.

Modes	Descriptions
in	Opens the file to read(default for ifstream)
out	Opens the file to write(default for ofstream)
binary	Opens the file in binary mode
app	Opens the file and appends all the outputs at the end
ate	Opens the file and moves the control to the end of the file
trunc	Removes the data in the existing file
nocreate	Opens the file only if it already exists
noreplace	Opens the file only if it does not already exist

https://www.edureka.co/blog/file-handling-in-cpp/